Avatar Concession Schedule Generator

Parameters

```
set.seed(8)

timeout <- 60 # Bargain timeout (bargain window)

n_star <- 23 # Expected number of concessions at timeout

theta <- 0.2 # Ending (terminal) rate of concessions per second of bargaining

# 0.2 = 5 seconds per step

kappa <- 0.075 # Concession rate convergence speed
```

Get Starting Rate

Get Y_0:

$$Y_0 = \theta + \frac{\kappa(n^* - \theta T)}{1 - e^{-\kappa T}}$$

```
# Starting rate of concessions per second of bargaining

Y0 <- theta + (kappa*(n_star - (theta*timeout)))/(1 - exp(-(kappa*timeout)))

Y0
```

```
[1] 1.034268
```

Define Rates

Define rate as a function of time:

$$\lambda(t) = \theta + (Y_0 - \theta)e^{-\kappa t}$$

With CDF equal to the expected number of concessions at time t:

$$\mathbb{E}[N_t] = \Lambda(t) = \int_0^t \lambda(s)\, ds = \theta t + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa t})$$

```
lambda <- function(time) {

  theta + (Y0 - theta) * exp(-(kappa*time))

}

LAMBDA <- function(time) {

  theta * time + ((Y0 - theta) / kappa) * (1 - exp(-kappa * time))

}

# Verify expectations E(N_t) = n_star

LAMBDA(timeout)
```

```
[1] 23
```

```
LAMBDA(timeout) == n_star
```

```
[1] TRUE
```

Define Theoretical Densities

Define Segments

```r
# Split total time in equally spaced segments

segments <- c()

Nsegments <- 5

for (i in 1:(Nsegments+1)) {

  segments[i] <- timeout / Nsegments * (i-1)

}

segments
```

```
[1]  0 12 24 36 48 60
```

Estimate Frequencies and Densities

```r
frequencies <- c()

densities <- c()

for (i in 1:length(segments[-1])) {

  frequencies[i] <- LAMBDA(segments[i+1]) - LAMBDA(segments[i])

  densities[i] <- frequencies[i] / LAMBDA(segments[length(segments)])

}

frequencies
```

```
[1] 9.001065 5.083793 3.491149 2.843628 2.580366
```

```r
sum(frequencies)
```

```
[1] 23
```

```
densities
```

```
[1] 0.3913507 0.2210345 0.1517891 0.1236360 0.1121898
```

```
sum(densities)
```

```
[1] 1
```

```
densitySD <- sqrt(densities*(1-densities)/n_star) # Binomial SD
densitySD
```

```
[1] 0.10176600 0.08652173 0.07481837 0.06863582 0.06580712
```

```
# Calibrate absolute frequency deviations per segment (allowed ranges)

scalingSD <- 0.6

cat("Maximum allowed frequencies: ", frequencies + densitySD * n_star * scalingSD, "\n\n")
```

```
Maximum allowed frequencies:  10.40544 6.277793 4.523642 3.790802 3.488504
```

```
cat("Expected frequencies: ", frequencies, "\n\n")
```

```
Expected frequencies:  9.001065 5.083793 3.491149 2.843628 2.580366
```

```
cat("Minimum allowed frequencies: ", frequencies - densitySD * n_star * scalingSD, "\n\n")
```

```
Minimum allowed frequencies:  7.596694 3.889793 2.458655 1.896454 1.672227
```

```
cat("Feasible upper: ", floor(frequencies + densitySD * n_star * scalingSD), "\n\n")
```

```
Feasible upper:  10 6 4 3 3
```

```
cat("Feasible lower: ", ceiling(frequencies - densitySD * n_star * scalingSD), "\n\n")
```

```
Feasible lower:  8 4 3 2 2
```

```
cat("Upper diff: ", floor(frequencies + densitySD * n_star * scalingSD) - frequencies, "\n\n
```

```
Upper diff:  0.998935 0.9162073 0.5088513 0.156372 0.4196343
```

```
cat("Lower diff: ", frequencies - ceiling(frequencies - densitySD * n_star * scalingSD), "\n
```

```
Lower diff:  1.001065 1.083793 0.4911487 0.843628 0.5803657
```

Empirical

```
get_empirical <- function(schedule, segments) {

  emprirical <- c()

  for (i in segments) {
    density <- sum(schedule <= i) / length(schedule)

    emprirical <- c(emprirical, density)
  }

  error <- abs(densities - diff(c(0, emprirical)))

  return(list(error=error, density=diff(c(0, emprirical))))
}
```

Draw Samples

NHPP with Ogata Thinning

Sample from a non-homogenous poisson process using Ogata thinning algorithm:

```
rejectionCount <- 0 # Discarded draws, save for diagnostics

scheduleList <- list()

densityMatrix <- matrix(nrow=0, ncol=Nsegments) # Save for diagnostics

# Get upper bound rate, include rate at time=0 in case rate is decreasing
```

```
M <- max(lambda(timeout), lambda(0), theta)

while (length(scheduleList) < 500) {

  t <- 0

  schedule <- c()

  while (t < timeout) {

  interval <- rexp(1, rate=M)

  nextTime <- t + interval

  if (nextTime > timeout) break

  if (runif(1) <= lambda(nextTime)/M) {

    schedule <- c(schedule, nextTime)

  }

  t <- nextTime

  }

  # Add a random initial delay to avoid jumpy start to the bargain

  schedule <- schedule + runif(1, min=1.5, max=3)

  # Filter out of range draws (after adding extra initial delay)

  schedule <- schedule[schedule < timeout]

  # Filter draws that deviate from the theoretical density

  empirical <- get_empirical(schedule, segments[-1])

  distributionFilter <- sum(empirical$error > scalingSD*densitySD) == 0

  # Filter draws that deviate from the expected count (even split equilibrium ensurance)
```

```
  scheduleLengthFilter <- 20 < length(schedule) & length(schedule) < 26
  if (distributionFilter & scheduleLengthFilter) {
    scheduleList[[length(scheduleList) + 1L]] <- schedule
    densityMatrix <- rbind(densityMatrix, empirical$density)
  } else {
    rejectionCount <- rejectionCount + 1
  }
}
```

```
#saveRDS(scheduleList, "ConcessionScheduleList.rds")
```

Check Output

Check output is well behaved

```
cat("Number of rejected draws: ", rejectionCount, "\n\n")
```

```
Number of rejected draws:  37973
```

```
cat("Theoretical densities: ", densities, "\n\n")
```

```
Theoretical densities:  0.3913507 0.2210345 0.1517891 0.123636 0.1121898
```

```
cat("Empirical (mean) densities: ", colMeans(densityMatrix), "\n\n")
```

```
Empirical (mean) densities:  0.3904929 0.224673 0.1558537 0.1175437 0.1114367
```

```
cat("Theoretical SDs: ", densitySD, "\n\n")
```

```
Theoretical SDs:  0.101766 0.08652173 0.07481837 0.06863582 0.06580712
```

```
cat("Empirical SDs: ", apply(densityMatrix, 2, sd), "\n\n")
```

```
Empirical SDs:  0.03127602 0.02895061 0.02291162 0.02415957 0.02288469
```

```
cat("Empirical max: ", apply(densityMatrix, 2, max), "\n\n")
```

```
Empirical max:  0.44 0.2727273 0.1904762 0.16 0.1428571
```

```
cat("Empirical min: ", apply(densityMatrix, 2, min), "\n\n")
```

```
Empirical min:  0.3333333 0.173913 0.12 0.08333333 0.08
```

```
cat("Empirical range: ", apply(densityMatrix, 2, max) - apply(densityMatrix, 2, min), "\n\n")
```

```
Empirical range:  0.1066667 0.09881423 0.07047619 0.07666667 0.06285714
```

```
evenSplitTime <- c()

totalConcessions <- c()


for (i in scheduleList) {

  evenSplitTime <- c(evenSplitTime, i[20])

  totalConcessions <- c(totalConcessions, length(i))

}
```

```
cat("Even split time, Mean: ", mean(evenSplitTime), " SD: ", sd(evenSplitTime), " Max: ", max(evenSplitTime), " Min: ", min(evenSplitTime), "\n\n")
```

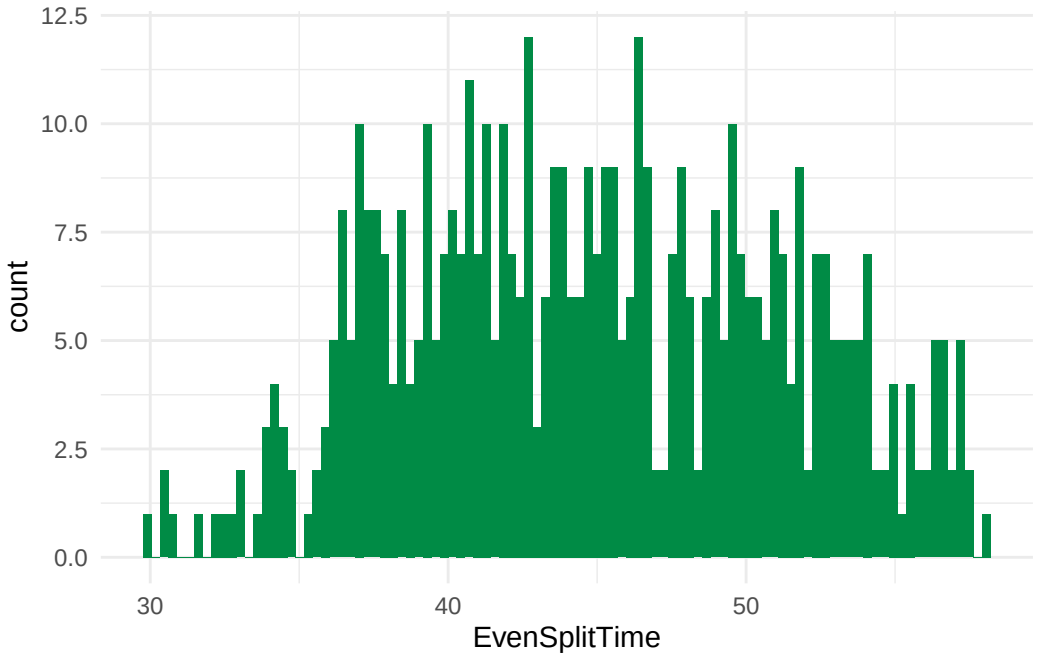
```
Even split time, Mean:  44.95284  SD:  6.297667  Max:  58.06332  Min:  29.90604
```

```
cat("Total concessions, Mean: ", mean(totalConcessions), " SD: ", sd(totalConcessions), " Max: ", max(totalConcessions), " Min: ", min(totalConcessions), "\n\n")
```

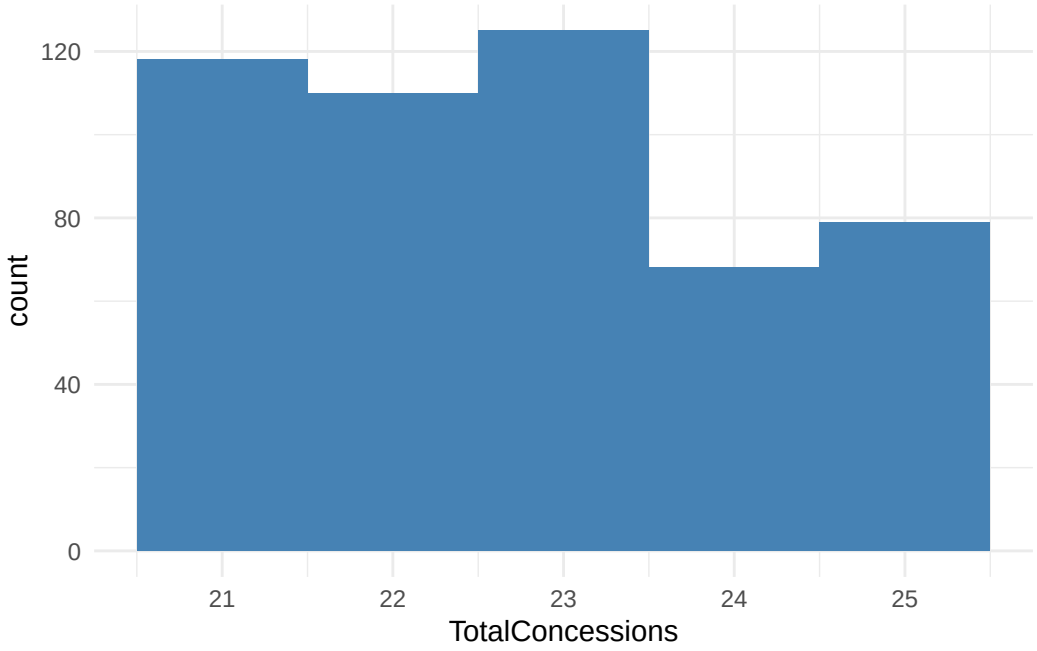
```
Total concessions, Mean:  22.76  SD:  1.370458  Max:  25  Min:  21
```

```r
df <- data.frame(EvenSplitTime = evenSplitTime, TotalConcessions = totalConcessions)

ggplot(df, aes(x=EvenSplitTime))+
  geom_histogram(bins=100, fill="springgreen4")+
  theme_minimal()
```




```r
ggplot(df, aes(x=TotalConcessions))+
  geom_histogram(bins=5, fill="steelblue")+
  theme_minimal()
```



Export to JSON

```
# Quantize helper (~float32-ish precision in seconds)
#q <- function(x, digits = 4) round(as.numeric(x), digits)

#schedules_json <- lapply(seq_along(scheduleList), function(i) {
#  list(
#    totalConcessions = totalConcessions[i],
#    timeToEquilibrium = evenSplitTime[i],
#    timestamps = q(scheduleList[[i]], 4)  # e.g., 12.3456
#  )
#})

#write_json(
#  schedules_json,
#  path = "ConcessionSchedules.json",
#  auto_unbox = TRUE,
#  pretty = FALSE
#)
```